STI-DBSCAN: Clustering spatial-time interval data and utilizing it for waze accident data

Vitória Guardieiro

Fundação Getulio Vargas

03/12/2018

Introduction

- A ideia deste projeto é desonvolver um algoritmo capaz de agrupar dados espaciais que se mantém em um intervalo de tempo
- Entretando, ao invés de utilizarmos como parâmetro o número de grupos que queremos, utilizaremos distâncias espaciais e temporais

Motivation and Data

- A motivação é a disponibilidade de certos dados de alertas do aplicativo Waze
- Mais especificamente, utilizaremos os alertas sobre acidentes
- Como a maioria dos outros tipos de alertas do aplicativo, o de acidentes funciona da seguinte maneira:
  - Um usuário pode adicionar a qualquer momento um alerta
  - Assim que colocado, um alerta fica um certo intervalo de tempo ativo no aplicativo
  - Outros usuários podem verificar o alerta, fazendo que este fique ativo por mais tempo

Motivation and Data

- Entretanto, temos dois principais problemas nos alertas:
  - Como cada usuário pode adicionar um alerta, é possível que haja mais de um alerta ao mesmo tempo para um mesmo acidente
  - Como cada alerta fica um certo intervalo de tempo, é possível que esse intervalo seja menor que o intervalo em que o acidente atrapalhou o trânsito na via, o que pode fazer com que um novo alerta seja adicionado posteriormente sobre o mesmo acidente

Motivation and Data

- Nosso objetivo é estimar a quantidade real de acidentes que ocorreram
- Assim, desenvolveremos um algoritmo que agrupe os alertas pelas distâncias espacial e temporal entre deles, de forma a encontrar os alertas que representam o mesmo acidente

DBSCAN

- Este foi o algoritmo base para desenvolver o meu próprio
- Não recebe como parâmetro a quantidade de grupos esperado, mas um parâmetro de distância e outro de quantidade mínima de pontos em um cluster

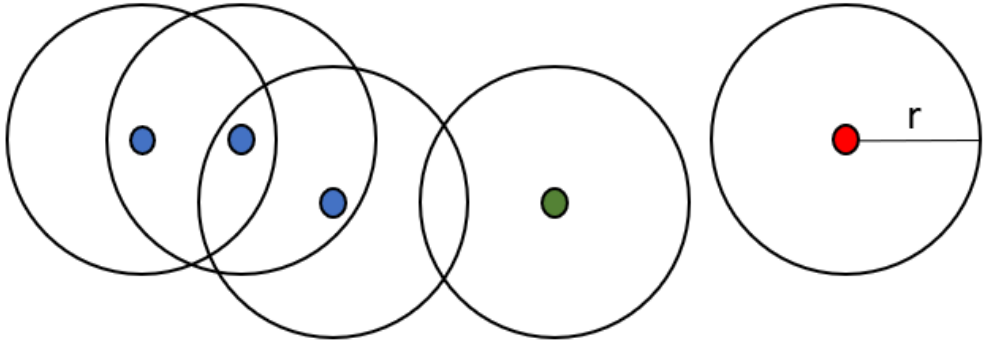


Figure 1: Exemplo de agrupamento com DBSCAN e parâmetro de distância r e quantidade mínima 1

DBSCAN

Algoritmo 1: DBSCAN

Entrada: D, ϵ, min

Saída: Número esperado de nodos atingidos

```
1  início
2      C = 0
3      para cada ponto P ∈ D não visitado faça
4          marco P como visitado
5          vizinhança = Querry(P, ε)
6          se tamanho(vizinhança) < min então
7              marco P como NOISE
8              senão
9                  C = C+1
10                 expandirCluster(P, vizinhança, C, ε, min)
11             fim
12         fim
13     fim
14 fim
```

DBSCAN

Algoritmo 2: EXPANDIRCLUSTER

Entrada: P, vizinhança, C, ϵ, *min*

```
1  início
2      adiciona P ao cluster C
3      para cada ponto P' ∈ vizinhança faça
4          se P' não foi visitado então
5              marco P' como visitado
6              vizinhança' = Querry(P', ε)
7              se tamanho(vizinhança') >= min então
8                  vizinhança = união de vizinhança com vizinhança'
9              fim
10         fim
11         se P' não é membro de nenhum cluster então
12             adiciona P' ao cluster C
13         fim
14     fim
15 fim
```

DBSCAN

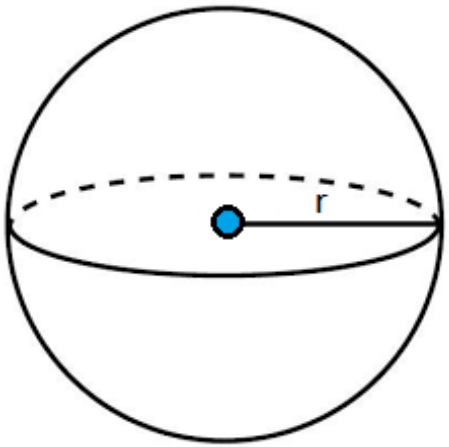
Algoritmo 3: QUERRY

Entrada: P, ϵ

Saída: Vizinhança de P

```
1 início
2 |   retorna todos os pontos na ϵ-vizinhança de P (incluindo P)
3 fim
```

Modelling the data



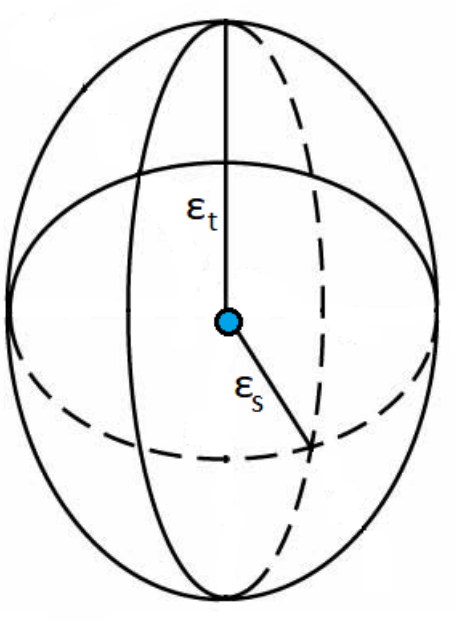
A primeira tentativa de modelar meus dados foi utilizar o *dbscan* sem alterações calculando um raio para a esfera espaço-temporal onde o centro seria o tempo inicial do alerta.

Modelling the data

Essa estratégia apresentou dois problemas principais:

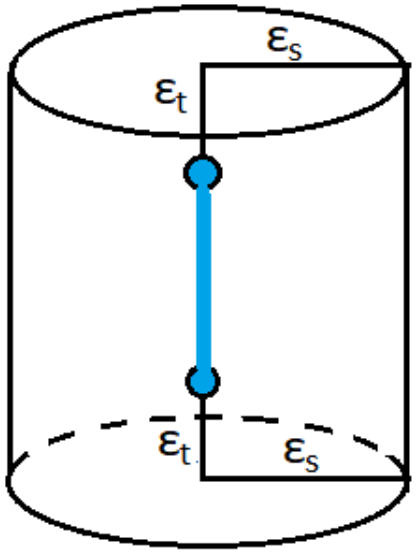
- Dar mais importância à distância espacial ou à temporal dependendo dos dados
- Não funcionar bem para alertas que ficaram ativos por muito tempo

Modelling our data



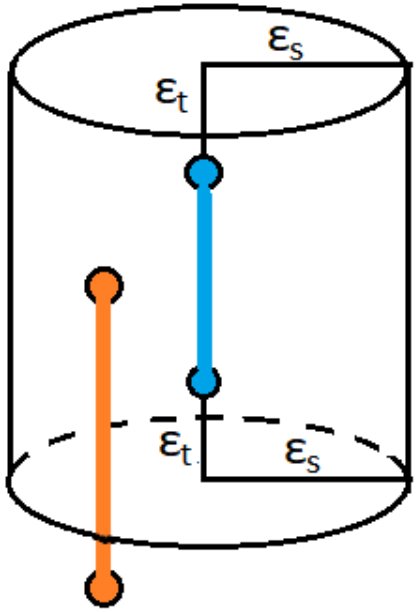
A segunda tentativa foi alterar a função *Querry* do *dbscan* de modo que fosse calculado um elipsoide ao redor de cada ponto. Assim, resolveria o primeiro problema, mas não o segundo.

Modelling our data



Na terceira tentativa, modelei a função *Querry* como um cilindro cujo raio é o parâmetro para a distância espacial e a altura é o seguimento que começa no instante inicial do alerta menos o parâmetro para a distância temporal e termina no instante final mais o mesmo parâmetro.

Modelling our data

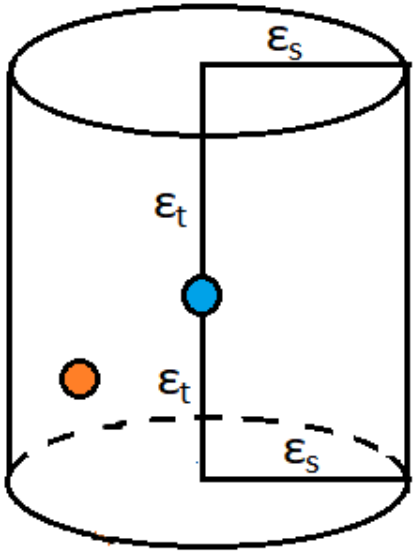


Diferentemente das outras estratégias, onde procurávamos por todos os pontos que estavam dentro do objeto, nesta procuramos pelos seguimentos de reta que interseptam o cilindro.

Modelling the data

Essa estratégia requer que sejam armazenadas duas informações sobre o tempo para alerta, o inicial e o final, enquanto que as estratégias anteriores requiriam apenas do inicial, mas isso é compensado por resolver ambos os problemas que tinha na primeira abordagem.

ST-DBSCAN



Após planejar a terceira estratégia, encontrei um algoritmo chamado *ST-DBSCAN* que justamente realiza essa *Querry* diferenciada do *dbscan* utilizando cilindro, só que novamente funciona apenas para dados pontuais e não para casos intervalares.
Utilizei ele como base para a implementação do meu em Python.

Modelling the data

Assim, os algortimos ficam:

Algoritmo 4: STI-DBSCAN

Entrada: C, $\epsilon_s, \epsilon_t, min = 1$

Algoritmo 5: EXPANDIRCLUSTER

Entrada: l, vizinhança, C, $\epsilon_s, \epsilon_t, min = 1$

Algoritmo 6: QUERRY

Entrada: l, ϵ_s, ϵ_t

Saída: Alertas na vizinhança estipulada de l

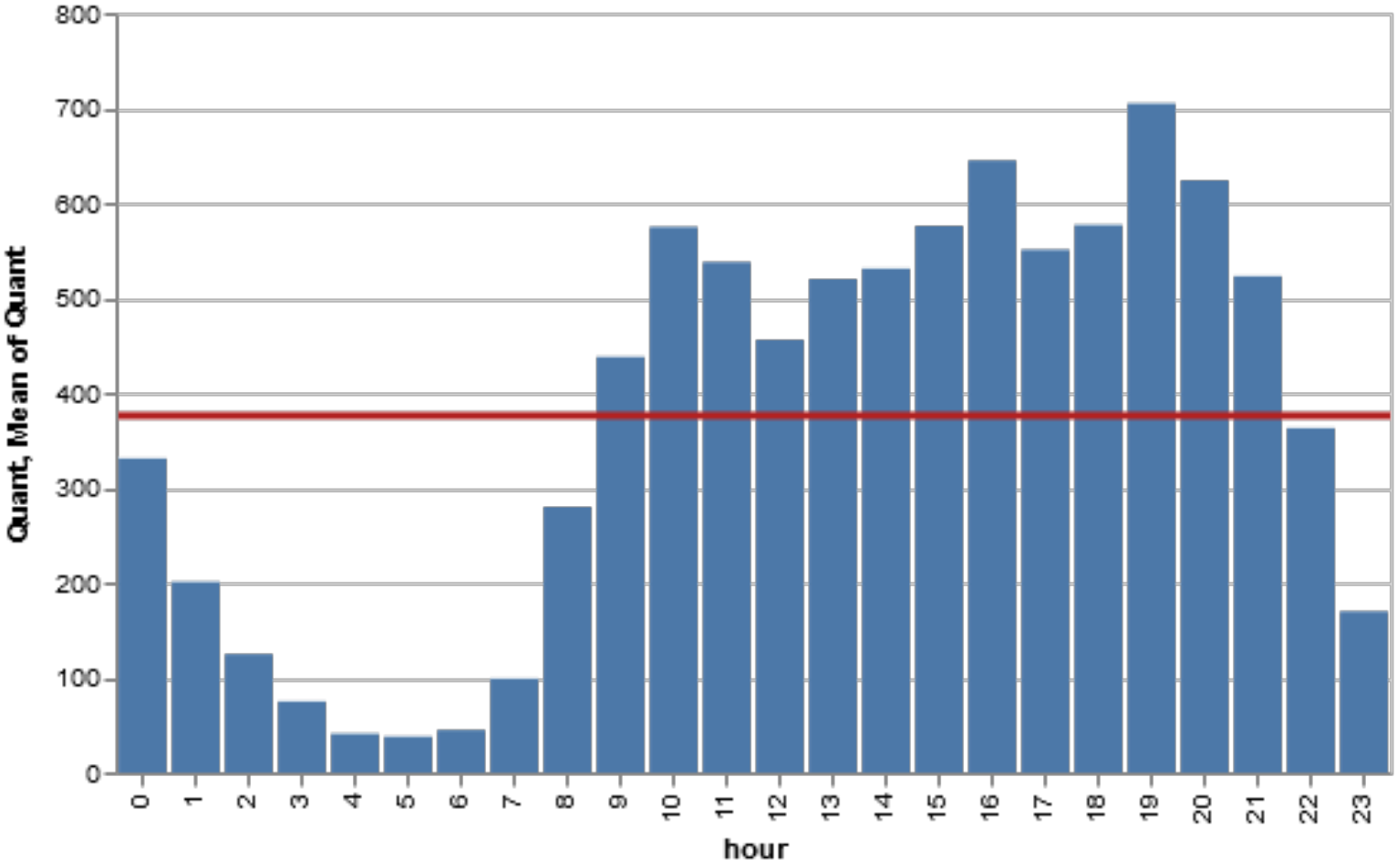
```
1 início
2     retorna todos os intervalos que intersectam o cilindro gerado
        pelo segmento l
3 fim
```

Results with data

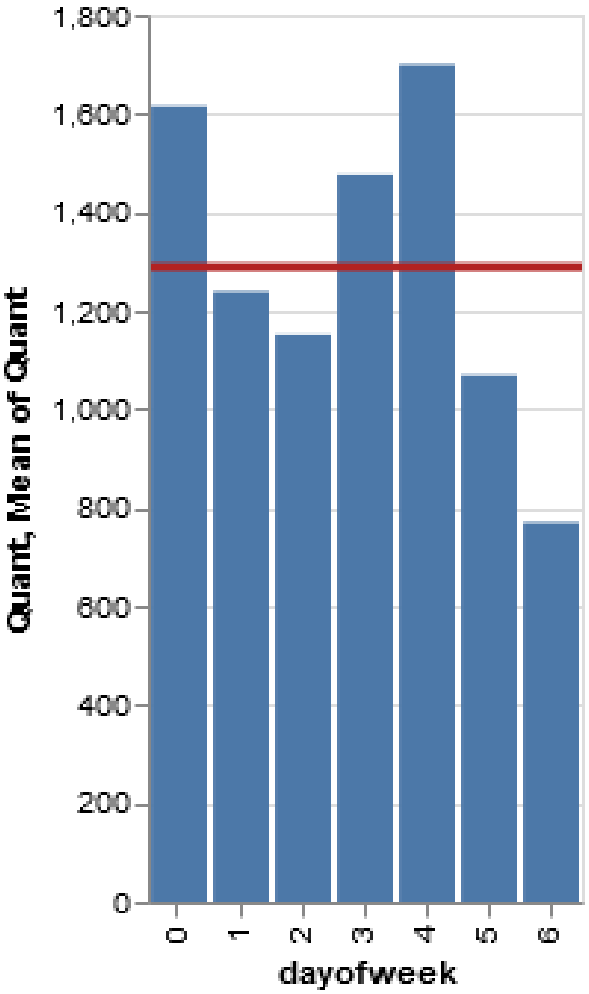
ϵ_s	100 m
ϵ_t	3600 s
Local	São Paulo - SP
Intervalo de tempo	01/11/2018 - 30/11/2018
Tempo de leitura	126.7162253856659 s
Tempo de processamento	735.0101065635681 s
Tempo total decorrido	861.726331949234 s
Total de alertas	11211
Total de acidente	9027

Results with data

Acidentes totais por hora



Results with data

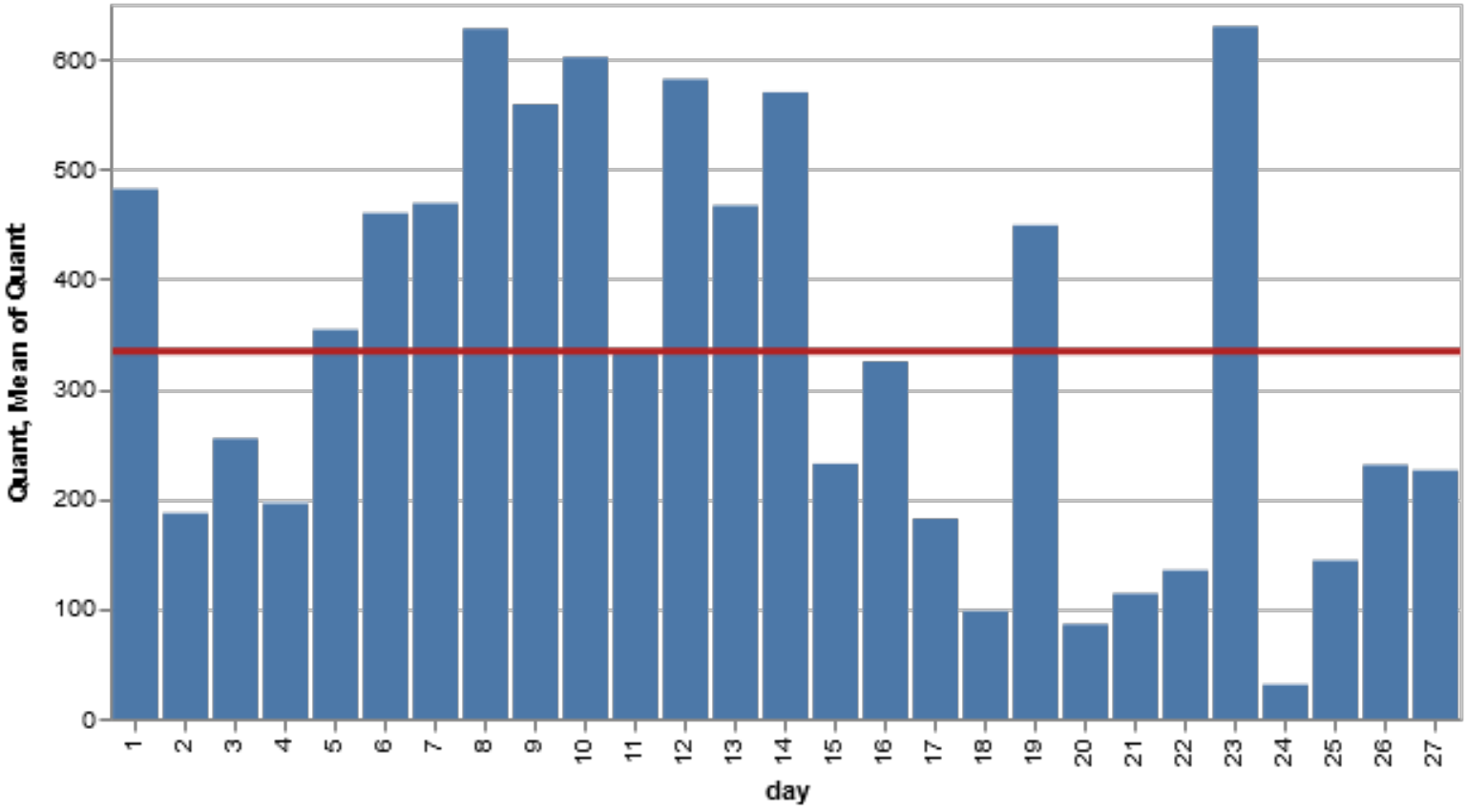


Acidentes totais por dia da semana

0	Segunda
1	Terça
2	Quarta
3	Quinta
4	Sexta
5	Sábado
6	Domingo

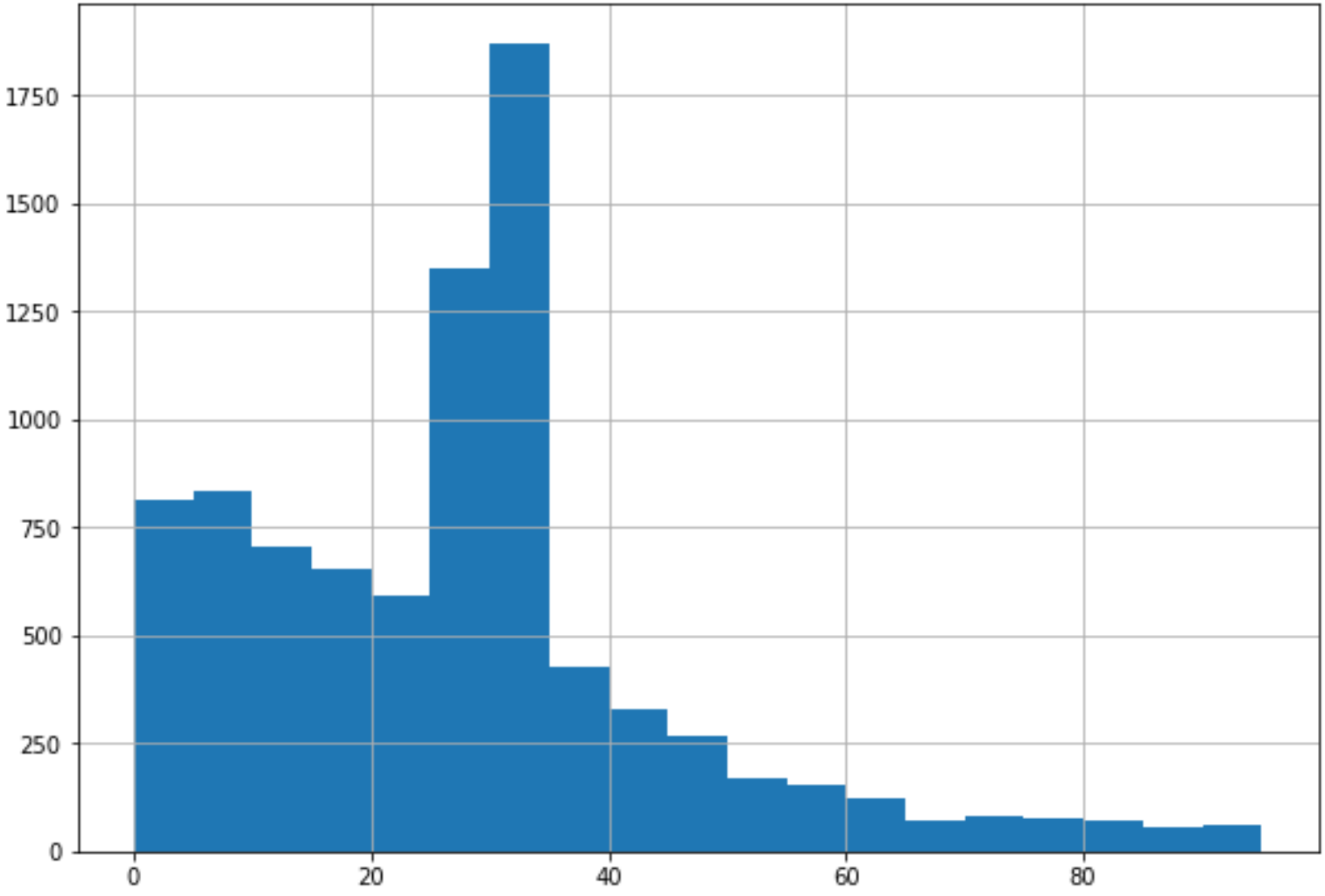
Results with data

Acidentes por dia

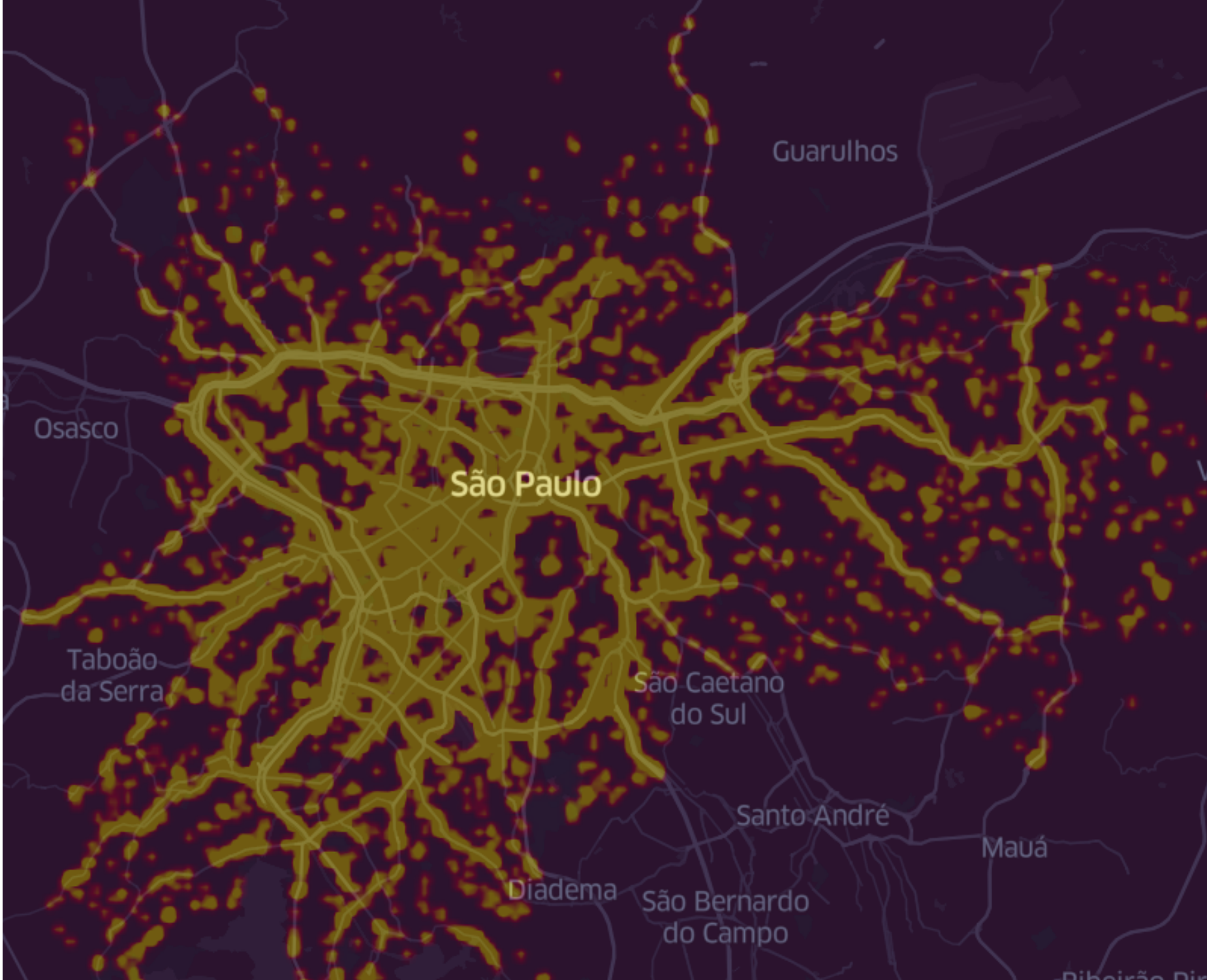


Results with data

Duração dos acidentes na via



Guarulhos
Osasco
São Paulo
Taboão
da Serra
São Caetano
do Sul
Santo André
Mauá
Diadema
São Bernardo
do Campo


Guarulhos
Osasco
São Paulo
Taboão
da Serra
São Caetano
do Sul
Santo André
Mauá
Diadema
São Bernardo
do Campo

References

1. Joel Grus
   Data Science from Scratch: First Principles with Python
2. Derya Birant, Alp Kut
   ST-DBSCAN: An algorithm for clustering spatial–temporal data